

Laboration 9 – Gör IoT-flödet observerbart

Uppgift

Kör referensdemon och instrumentera sedan ett eget eller det givna IoT-flödet. Ni ska kunna visa ett normalfall och ett avsiktligt fel med strukturerad loggning, minst ett mätetal och en reproducerbar felsökningsguide.

Git är valfritt för den fristående demon. Ändringar i IoT-caset sparas i Git.

Terminalroller

- Terminal 1 – Server/gateway: Kör vald Python- eller C++-server och visar loggar
- Terminal 2 – Simulerad sensorenhet: Kör klienten i samma språkspår och kontrollanrop
- Terminal 3, valfri – Passiv observatör: Fångar endast trafik till port 8090

Använd terminal 3 bara om trafikfångsten ingår i gruppens bevis. Den representerar ingen extra tjänst.

Del 1 – Starta och förutsäg

Välj [Python-demot](#) eller [C++-demot](#). De är funktionellt likvärdiga och bedöms på samma resultat. Skriv före körningen vilka värden ni förväntar er efter ett klientscenario för:

- Accepterade mätningar
- Valideringsfel
- Okända resurser
- Totalt antal HTTP-begäranden

Pythonspåret startas så här i terminal 1:

```
cd dag_9/demo_observability_py
python3 server.py
```

Och i terminal 2:

```
cd dag_9/demo_observability_py
python3 client.py
```

C++-spåret byggs och körs i stället så här:

```
cd dag_9/demo_observability_cpp
cmake -S . -B build
cmake --build build
./build/observability_server
```

I terminal 2:

```
cd dag_9/demo_observability_cpp  
./build/observability_client
```

Del 2 – Korrelera ett fel

Hitta klienttraden och serverloggen med `request_id demo-invalid-1`. Dokumentera:

- HTTP-status
- Loggnivå och eventnamn
- Felorsak
- Svarstid
- Varför detta är ett klientfel och inte ett serverfel

Kontrollera att svaret innehåller samma `X-Request-ID`.

Del 3 – Kontrollera mätetal

```
curl http://127.0.0.1:8090/api/metrics  
curl http://127.0.0.1:8090/metrics
```

Öppna `http://127.0.0.1:8090/dashboard`. Jämför dashboarden med JSON-svaret och textformatet. Förklara varför tre vyer inte betyder tre oberoende mätningar.

Del 4 – Filtrera nätverkstrafik

Valfri men rekommenderad när Wireshark eller tcpdump finns.

Linux:

```
sudo tcpdump -i lo -nn -A 'tcp port 8090'
```

Wireshark:

```
Capture filter: tcp port 8090  
Display filter: tcp.port == 8090 && http
```

Kör klienten igen. Hitta anslutningens adresser, portar, metod och statuskod. Skriv vad som inte skulle vara läsbart om samma trafik gick över TLS.

Del 5 – Implementera en förbättring

Välj minst en:

- Lägg till en räknare `unknown_paths_total`
- Lägg till fältet `sensor_id` i loggen för accepterade mätningar
- Lägg till ett mätetal för senast accepterade mätningens ålder
- Lägg till en klientbegäran med egen händelsetyp och eget `request_id`
- Instrumentera en relevant händelse i gruppens IoT-case

Skriv definitionen innan ni kodar: namn, typ, enhet, när värdet ändras och vilken fråga det ska besvara.

Del 6 – Skriv och byt felsökningsguide

Använd [observationsprotokollet](#). Guiden ska innehålla:

1. Version och startläge
2. Exakta kommandon eller klicksteg
3. Testdata utan hemligheter
4. Förväntat och observerat resultat
5. Relevant `request_id`, loggrad och mätvärde
6. Återställningssteg

Byt guide med en annan grupp. Testgruppen får inte muntlig hjälp under första försöket. Korrigera guiden efter deras återkoppling.

Valbar fördjupning

- Beräkna felandel över ett dokumenterat intervall
- Lägg till varning när felandelen passerar en motiverad tröskel
- För ett meddelande-ID från MQTT-payload till HTTP-header och logg
- Jämför medel och max med en enkel 95:e percentil
- Exponera samma mätetal från ESP32-demots `statistics_task`

Arbetsresultat

- Strukturerad logg med stabila fält
- Minst ett definierat och verifierat mätetal
- Ett korrelerat normal- eller felförlopp
- Två oberoende beviskällor
- En guide som en annan grupp har provat
- Inga hemligheter i logg, testdata eller Git